

Table 1: Key capabilities of hybrid SQL databases

Features	Traditional SQL	Vector DB only	Hybrid SQL DB
Relational joins (e.g., SQL JOIN)	Yes	No	Yes
Vector similarity search	No	Yes	Yes
Text, graph, or time-series data	No	No	Yes
Business filters (e.g., WHERE)	Yes	No	Yes
Unified query language (SQL)	Yes	No	Yes
AI + analytics in one system	No	No	Yes

Table 2: Why use PostgreSQL for vectors instead of a specialised vector DB?

Vector DBs	PostgreSQL (with pgvector)
High-speed ANNs (artificial neural networks)	Good enough for low to medium workloads
Limited to vector search	Full SQL, joins, transactions, etc
Hosted-only or managed	Full control, open source, no vendor lock-in
Separate stack	Integrate into existing Postgres apps

- **Unstructured data:** Text, images, conversations, embeddings
- **Semantic tasks:** Finding ‘similar’ items, understanding intent, detecting patterns

Traditional SQL databases (like PostgreSQL, MySQL) are great at structured queries but can’t handle semantic search or AI directly. Dedicated vector databases can handle AI tasks, but don’t support SQL joins, filters, or business logic. Hybrid SQL databases give you the best of both.

PostgreSQL meets vectors: The rise of ‘pgvector’

pgvector is an open source extension that turns PostgreSQL into a fully capable vector database by allowing similarity search using operators like:

- Store embeddings generated from OpenAI, Hugging Face, or other LLMs
 - Perform similarity searches using inner product, cosine, or Euclidean distance
 - Leverage GIN, IVFFlat, or HNSW indexing (via integrations like pgvecto.rs) for efficient ANN search
- PostgreSQL is ideal for hybrid workloads: structured + unstructured, analytics + AI, all in one place.

Example: Semantic document search in PostgreSQL:

```
- Store document text and its embedding
CREATE TABLE documents (
  id serial PRIMARY KEY,
  title text,
  content text,
  embedding vector(1536) -- assuming embedding size
);
-- Query top 5 most similar documents
SELECT title, content
FROM documents
ORDER BY embedding <-> '[0.11, 0.93, ..., -0.44]'
LIMIT 5;

-- Query top 5 most similar documents
SELECT title, content
FROM documents
ORDER BY embedding <-> '[0.11, 0.93, ..., -0.44]'
LIMIT 5;
```

Real world use cases of hybrid SQL databases

Use case 1: Early detection of aircraft engine failure risk through semantic anomaly pattern matching

Problem: Aircraft engines generate massive amounts of structured telemetry data (temperature, pressure, vibration, fuel flow) and unstructured maintenance logs (technician notes, fault descriptions, pilot complaints). Most failure events are preceded by subtle signals across both data types, but:

- These signals are not always identical across aircraft.
- Descriptions vary (e.g., “hot start anomaly” vs “unusual exhaust temperature spike”).
- Traditional monitoring tools focus on hard thresholds, missing context-based risk indicators.

Solution: Vector-based semantic monitoring for engine anomalies using pgvector.

Goal: To proactively detect early warning signs of engine failure by matching current sensor anomalies and pilot/maintenance notes with historical incidents stored as semantic vectors.

How it works:

1. **Historical incident embedding:** Thousands of engine fault incidents (text logs + sensor signature descriptions) are embedded into vectors. Each vector encodes semantic meaning — including symptoms, causes, and resolutions.
2. **Real-time data matching:** A new engine report from the flight crew says: “Slight engine vibration during cruise, EGT slightly elevated but stable, auto throttle fluctuated briefly.” This report is embedded using OpenAI or AeroBERT. A pgvector similarity search identifies past cases with similar descriptions, even if the wording is different.